

Table 6: Summary without asterisk

Example	Part Before Asterisk	Part After Asterisk	Comments
const char*ptr	const	ptr	Const is associated with data type, so value is constant
char const*ptr	char const	ptr	Const is associated with data type, so value is constant
char*const ptr	char	const ptr	Const is associated with pointer, so pointer is constant
const char*const ptr	const char	const ptr	Const is associated with both data type & pointer so both are constant

```
satya@satya:~$ gcc pointer_const.c
pointer_const.c: in function 'main':
pointer_const.c:14:12: error: assignment of read-only variable 'ptr'
  ptr = &b; //Error :Now, ptr is pointing to value of the variable 'b'
             ^
satya@satya:~$
```

Figure 5: Output of the code snippet shown in Illustration 3

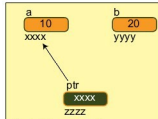


Figure 6: Pictorial representation of constant pointer usage

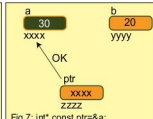


Figure 7: Pictorial representation showing value contained in the variable can be changed through the constant pointer

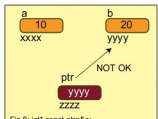


Figure 8: Pictorial representation showing constant pointer value can not be changed

```
the value is not constant
13
14     ptr = &b; //
Error:Now, ptr is pointing to
the value of the variable 'b'
15
16     return 0;
17
18 }
```

From the above example (Illustration 3), it is clear that in Line No 14 we tried to change the address of the pointer *ptr* to some other variable, but it is not possible. The output of the code snippet shown in Illustration 3 is given in Figure 5. Similarly, one can observe that in Line No 12, we are trying to change the value of the variable it is 'pointing to', which is possible.

This can be clearly understood by the pictorial representations given in Figures 6, 7 and 8.

Something to think about

Can we have both pointer to constant and constant pointer in a single statement?

Usage

We can find 'n' number of uses of these concepts in C as well as in the embedded C programming world. One

```
satya@satya:~$ gcc pointer_const.c
pointer_const.c: in function 'main':
pointer_const.c:14:12: error: assignment of read-only variable 'ptr'
  ptr = &b; //Error :Now, ptr is pointing to value of the variable 'b'
             ^
satya@satya:~$
```

Figure 9: Shows the usage of pointer to constant in strlen() library function

```
satya@satya:~$ gcc strlen_ptr.c
strlen_ptr.c: in function 'main':
strlen_ptr.c:14:12: error: assignment of read-only variable 'ptr'
  ptr = &b; //Error :Now, ptr is pointing to value of the variable 'b'
             ^
satya@satya:~$
```

Figure 10: Shows the usage of pointer to constant in strcmp() library function

such simple use of 'pointer to constant' is to find the string length of the given string without any attempt to modify the original string as shown in Example 1 (Figure 9). Example 2 gives an idea of using 'pointer to constant' in the *strcmp()* function (Figure 10).

A trick

There is a small trick to understand the difference between 'pointer to constant' and 'constant pointers' which is shown in Table 6.



Note: This trick is for all those new to the C programming world, who are confused with constant and pointers.

From the summary shown in Table 5, separate the part before asterisk(*) and the part after the asterisk(*) as given in Table 6, to clearly understand whether 'data' is constant or 'pointer' is constant. **END**

By: Satyanarayana Sampangi

The author is a member - Embedded software at Emertxe Information Technologies (<http://www.emertxe.com>). His area of interest lies in embedded C programming combined with data structures and microcontrollers. He likes to experiment with C programming and open source tools in his spare time to explore new horizons. He can be reached at satya@emertxe.com